

T2 Exercises – Fateme Firouznia

1. In fdisk, when using the p (print) command, there is a column labeled "sectors."

What does "sector" refer to in this context?

In the output of the p (print) command in fdisk, the "sectors" column shows how many disk sectors are allocated to each partition.

A sector is the smallest addressable unit on a disk. The disk stores data in very small blocks, and each block is one sector, typically 512 bytes or 4096 bytes in size.

When fdisk reports the number of sectors for a partition, it is expressing the partition's size in terms of these physical storage units rather than in megabytes or gigabytes.

This representation is used by partitioning tools and helps calculate the exact layout of the disk.

2. There are three types of servers mentioned.

Please summarize your partitioning recommendations for each:

- **Linux desktop systems**

Desktop machines mainly store user files, so the root and home partitions are separated.

Root typically needs **25–50 GB** for the OS and applications, while **/home takes most remaining space** for documents and personal data.

Swap is sized equal to RAM if hibernation is required; otherwise **2–4 GB** is enough.

/boot (about **500 MB–1 GB**) holds kernel files, and UEFI systems also need an **EFI partition** of **100–500 MB**.

- **Linux servers used for databases or web services with extensive logging**

The goal is to isolate data and prevent logs from filling the root filesystem.

Root usually needs **20–30 GB**, with a small swap (around **2–4 GB**).

Database files go in a dedicated partition such as **/var/lib/mysql**, often taking **40–60%** of the disk depending on expected growth.

Logs are placed in **/var/log** with **10–20 GB** to avoid filling root.

Web content may be placed under **/var/www**.

/boot (500 MB–1 GB) and an **EFI partition** (100–500 MB) are also included.

- **Linux servers used in university labs or staging environments where multiple users require individual home directories and personal storage space**

These systems host many users, so **/home** is separated and often assigned **20–30%** of the disk with quotas.

The root partition usually needs **30–50 GB** due to diverse software installations.

Swap is **4–8 GB** for occasional memory-intensive tasks.

/boot (about 1 GB) allows multiple kernels, and UEFI requires a **200–500 MB** EFI partition.

Shared class or project data is placed in **/data** or **/shared** (about **30–40%** of space).

If virtual machines are used, **/vm** or **/var/lib/libvirt** may take **20–30%** of disk capacity.

3. **How does an operating system run multiple applications when the total available RAM is less than the combined memory requirements?**

An operating system can run multiple applications even when total RAM is smaller than their combined memory requirements by using CPU scheduling and virtual memory mechanisms.

Processes are executed through **time-slicing**, where the scheduler rapidly switches the CPU between programs so each process receives short execution intervals. This allows multiple applications to appear as if they are running simultaneously.

Process data is loaded into RAM using **demand paging**, meaning that only the memory pages actively needed by a process are brought into physical memory.

Pages that are not currently in use remain on disk until they are accessed.

When RAM becomes insufficient, the kernel frees space by moving inactive pages to disk.

This occurs through **paging**, where individual pages are evicted to the swap area, or, in uncommon cases, **swapping**, where most of a process's memory is moved to disk. The swap area functions as overflow memory—much slower than RAM but essential for keeping the system operational when physical memory is full.

Because of demand paging, a process does not need all of its pages loaded into RAM to execute. Only the pages currently being accessed must reside in physical memory, while the rest stay on disk until required.

4. **What is the Translation Lookaside Buffer (TLB), and what role does it play in memory management?**

The Translation Lookaside Buffer (TLB) is a small, high-speed cache inside the CPU that stores recent translations from virtual addresses to physical addresses.

Every memory access issued by a process uses virtual addresses, and each must be translated through the page table.

Reading the page table from RAM on every access would be too slow.

The TLB speeds this up by caching the most recently used translations.

If the translation is found in the TLB (a TLB hit), the CPU can access memory immediately.

If it is not found (a TLB miss), the system must consult the page table in RAM, which takes longer.

The primary role of the TLB is to reduce the cost of address translation and significantly improve memory access performance.

5. What are a page, a virtual page, and a context switch? Additionally, How does increasing swap space affect context-switching performance? How does page size influence these effects?

A **page** is the smallest unit of memory management used by the operating system.

Programs work with **virtual pages**, which belong to a virtual address space and are mapped to physical page frames in RAM when needed. Pages that are not currently in use may be placed on disk if they are swapped out.

A **context switch** occurs when the CPU stops running one process and loads the state of another.

If the next process needs pages that are not in RAM and must be retrieved from swap, the context switch becomes slower due to the required disk I/O.

Increasing swap space does not make switching faster; it simply allows more processes to exist when RAM is full. However, it also increases the chance that active pages end up on disk, which slows down memory access and therefore slows context switching.

Page size also influences performance. Larger pages reduce the number of page table entries and improve TLB hit rates, but if swapping occurs, larger pages take longer to load from disk. Smaller pages create more mappings and may increase memory-management overhead during context switches.

6. what is a huge page?

Please explain its purpose and when it is used.

A huge page is a memory page that is significantly larger than a normal page, typically 2 MB or 1 GB instead of the standard 4 KB.

Huge pages reduce the total number of pages the OS must manage, which decreases page table size and improves TLB hit rates. This reduces the overhead of address translation and leads to faster memory access.

Huge pages are commonly used in workloads that handle large, contiguous memory regions, such as database servers, virtual machines, high-performance computing applications, and systems that frequently access large data structures. In these cases, huge pages help improve performance by lowering memory-management overhead.

7. What is memory fragmentation in RAM, and what problems can it cause?

Memory fragmentation occurs when free RAM exists but is divided into many small, non-contiguous blocks.

In this situation, the operating system may be unable to allocate a large continuous region of memory even though the total amount of free memory is sufficient. This is known as external fragmentation. Internal fragmentation can also occur when a process is given more memory than it actually needs, leaving part of the allocated block unused.

Fragmentation can cause several problems. It reduces overall memory efficiency and may lead to allocation failures for applications that require large contiguous memory regions. It can also degrade performance because the OS may need to perform memory compaction or rely more heavily on swapping, both of which introduce significant overhead.